

Unit-5 : Deployment of Models

Asst. Prof. Vasanti Chavda

Programming for AI (03012402PC01)

Artificial Intelligence and Data Science Department

Content

1. What is Model Deployment?	3
2. Deploying Models with Flask	4
3. Flask Deployment Example	5
4. Deploying Models with Django	6
5. Django Implementation Overview	7
6. Understanding Cloud Platforms	8
7. Deploying AI Models on AWS	9
8. AWS SageMaker Deployment Process	10
9. Deploying on Google Cloud Platform	11
10. Vertex AI Deployment Workflow	12
11. Deployment Best Practices	13

What is Model Deployment?

Model deployment is the process of integrating a trained machine learning model into a production environment where it can receive input data and return predictions.

Key Objectives:

- Make models accessible to end users or applications
- Ensure scalability and reliability
- Monitor performance and maintain accuracy
- Enable real-time or batch predictions

Deploying Models with Flask

Flask is a lightweight Python web framework perfect for creating simple REST APIs for ML models.

Basic Flask Deployment Steps:

- **Save your model:** Use `pickle` or `joblib`
- **Create Flask app:** Set up routes and endpoints
- **Load model:** Import the saved model
- **Create prediction endpoint:** Accept input and return predictions
- **Run the server:** Deploy locally or on a cloud server

Flask Deployment Example

```
from flask import Flask, request, jsonify
import pickle

app = Flask(__name__)

# Load the trained model
with open('model.pkl', 'rb') as f:
    model = pickle.load(f)

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    prediction = model.predict([data['features']])
    return jsonify({'prediction': prediction.tolist()})

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Deploying Models with Django

Django is a full-featured Python web framework suitable for complex, production-grade applications.

Django Deployment Advantages:

- Built-in admin interface for model management
- Robust ORM for database operations
- Built-in security features (CSRF, SQL injection protection)
- Better suited for large-scale applications
- Authentication and user management out of the box

When to use Flask:

Simple APIs, microservices, quick prototypes

When to use Django:

Complex apps, multiple models, user management

Django Implementation Overview

Key Components:

- **Views:** Handle request logic and call the model
- **Models:** Store prediction history in database
- **Serializers:** Convert data formats (with Django REST Framework)
- **URLs:** Map endpoints to views

Django REST Framework: Extends Django with powerful features for building REST APIs including serialization, authentication, and browsable API interface.

Setup: `pip install django
djangorestframework`

Understanding Cloud Platforms

Why Deploy on Cloud?

- **Scalability:** Handle varying loads automatically
- **Reliability:** High availability and fault tolerance
- **Cost-effective:** Pay only for what you use
- **Global reach:** Deploy closer to users worldwide
- **Managed services:** Less infrastructure management

Major Cloud Providers:

AWS (Amazon Web Services)

Market leader, extensive services

Google Cloud Platform

Strong in AI/ML, competitive pricing

Deploying AI Models on AWS

AWS ML Deployment Options:

- **Amazon SageMaker:** End-to-end ML platform (recommended)
- **AWS Lambda:** Serverless functions for lightweight models
- **Amazon EC2:** Virtual servers with full control
- **Amazon ECS/EKS:** Container orchestration
- **AWS Elastic Beanstalk:** PaaS for web applications

Amazon SageMaker is the preferred choice for ML deployment as it provides built-in model hosting, automatic scaling, and model monitoring.

AWS SageMaker Deployment Process

Step-by-Step Deployment:

- **1. Prepare Model:** Train or import your model
- **2. Create Model Artifact:** Package with dependencies
- **3. Upload to S3:** Store model files in S3 bucket
- **4. Create Model:** Define model in SageMaker
- **5. Create Endpoint Configuration:** Specify instance type
- **6. Deploy Endpoint:** Launch the inference endpoint
- **7. Invoke Endpoint:** Make predictions via API

Instance Types: ml.t2.medium (dev/test) to
ml.p3.8xlarge (production)

Deploying on Google Cloud Platform

GCP ML Deployment Options:

- **Vertex AI:** Unified ML platform (similar to SageMaker)
- **Cloud Run:** Serverless containers
- **Google Kubernetes Engine (GKE):** Managed Kubernetes
- **Cloud Functions:** Serverless functions
- **Compute Engine:** Virtual machines

Vertex AI combines AutoML and custom training, making it easy to deploy models with managed infrastructure and automatic scaling.

Vertex AI Deployment Workflow

Deployment Steps:

- **1. Upload Model:** Upload to Model Registry
- **2. Configure Resources:** Select machine type and GPU
- **3. Create Endpoint:** Set up prediction endpoint
- **4. Deploy Model:** Associate model with endpoint
- **5. Monitor & Scale:** Use built-in monitoring tools

GCP Advantages:

- TensorFlow integration
- BigQuery ML support
- Competitive pricing

Best For:

- Deep learning models
- Large-scale data processing
- Real-time predictions

Deployment Best Practices

Essential Practices:

- **Version Control:** Track model versions and rollback capability
- **Monitoring:** Track latency, accuracy, and resource usage
- **Logging:** Comprehensive logging for debugging
- **Security:** API authentication, HTTPS, input validation
- **CI/CD Pipeline:** Automate testing and deployment
- **A/B Testing:** Test new models before full rollout
- **Model Retraining:** Schedule periodic retraining

Remember: Deployment is not the end - continuous monitoring and maintenance are crucial for production models!

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

